Lab 10 - Game - Room

```java
package Lab10.Game;

public class Room
{
    private final String roomName;
    private final String roomPuzzle;
    private final String answer;

    /// <b>{@code INTERNAL}</b>
    boolean solved = false;

    public Room(String roomName, String roomPuzzle, String answer)
    {
        this.roomName = roomName;
        this.roomPuzzle = roomPuzzle;
        this.answer = answer;
    }

    public boolean validSolution(String answer) { return
this.answer.equalsIgnoreCase(answer); }

    /* GETTERS */

    public String getRoomName() { return roomName; }

    public String getRoomPuzzle() { return roomPuzzle; }

    public String getAnswer() { return answer; }

    /// <b>{@code INTERNAL}</b>
    void setSolved() { this.solved = true; }

    /// <b>{@code INTERNAL}</b>
    boolean isSolved() { return this.solved; }
}
```

Lab 10 - Game - VirtualEscapePuzzle

```java
package Lab10.Game;

import java.util.ArrayList;
import java.util.Optional;

public class VirtualEscapePuzzle
{
    private final ArrayList<Room> rooms = new ArrayList<>();

    public Room createRoom(String roomName, String roomPuzzle, String answer)
    {
        return new Room(roomName, roomPuzzle, answer);
    }

    public void addRoom(Room room) { rooms.add(room); }

    public Optional<Room> findRoomByName(String name)
    {
        return rooms.stream().filter(room ->
room.getRoomName().equalsIgnoreCase(name)).findFirst();
    }

    public RoomSolvedError solveRoom(String roomName, String answer)
    {
        return findRoomByName(roomName).map(room -> {
            boolean solved = room.validSolution(answer);
            if (solved) room.setSolved();
            return solved ? RoomSolvedError.NO_ERROR : RoomSolvedError.WRONG_ANSWER;
        }).orElse(RoomSolvedError.ROOM_NOT_FOUND);
    }

    public ArrayList<Room> getUnsolvedRooms()
    {
        return rooms.stream()
                .filter(room -> !room.isSolved())
                .collect(ArrayList::new, ArrayList::add, ArrayList::addAll);
    }

    public ArrayList<Room> getSolvedRooms()
    {
        return rooms.stream().filter(Room::isSolved).collect(ArrayList::new,
ArrayList::add, ArrayList::addAll);
    }

    public ArrayList<Room> getAllRooms() { return rooms; }

    /* UTILITIES */

    public enum RoomSolvedError
    {
        WRONG_ANSWER,
        ROOM_NOT_FOUND,
```

```
        NO_ERROR;
    }
}
```

Lab 10 - ConsoleUtils

```
package Lab10;

import javax.swing.*;

public class ConsoleUtils
{
    private static boolean modernMode = true;

    public static void setModernMode(boolean modernMode) { ConsoleUtils.modernMode =
modernMode; }

    /* SYMBOLS */

    private static String h() { return modernMode ? "─" : "-"; }

    private static String v() { return modernMode ? "│" : "|"; }

    private static String tl() { return modernMode ? "┌" : "+"; }

    private static String tr() { return modernMode ? "┐" : "+"; }

    private static String bl() { return modernMode ? "└" : "+"; }

    private static String br() { return modernMode ? "┘" : "+"; }

    private static String ml() { return modernMode ? "├" : "+"; }

    private static String mr() { return modernMode ? "┤" : "+"; }

    /* BOX */

    public static void printTopBorder(int width) { println(tl() + repeat(h(), width)
+ tr()); }

    public static void printMiddleBorder(int width) { println(ml() + repeat(h(),
width) + mr()); }

    public static void printBottomBorder(int width) { println(bl() + repeat(h(),
width) + br()); }

    public static void printCenteredLine(String text, int width) { println(v() +
centerText(text, width) + v()); }

    public static void printLeftLine(String text, int width)
    {
        println(v() + " " + padRight(text, width - 2) + " " + v());
    }

    /* COMPONENTS */

    public static void printBox(String title, String[] lines, int width)
    {
```

```
        printTopBorder(width);

        if (title != null && !title.isEmpty())
        {
            printCenteredLine(title, width);
            printMiddleBorder(width);
        }

        for (String line : lines)
        {
            printLeftLine(line, width);
        }

        printBottomBorder(width);
    }

    /* GENERAL */

    public static String repeat(String s, int count) { return s.repeat(Math.max(0,
count)); }

    public static String centerText(String text, int width)
    {
        if (text.length() >= width) return text;

        int padding = width - text.length();
        int left = padding / 2;
        int right = padding - left;

        return " ".repeat(left) + text + " ".repeat(right);
    }

    public static String padRight(String text, int width)
    {
        if (text.length() >= width) return text;
        return text + " ".repeat(width - text.length());
    }

    /* INTERNAL */

    private static void println(String s) { System.out.println(s); }

    public static void oneLineSpace() { System.out.println(); }
}```


# Lab 10 - Main

```java
package Lab10;

import Lab10.Game.Room;
import Lab10.Game.VirtualEscapePuzzle;
```
```

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Optional;
import java.util.Scanner;

public class Main
{
    private static final Scanner IN = new Scanner(System.in);
    private static final int HELP_BOX_WIDTH = 30;
    private static final int GAME_BOX_WIDTH = 60;

    public static void main(String[] args)
    {
        // --no-modern for old consoles
        if (Arrays.stream(args).anyMatch(arg -> arg.equalsIgnoreCase("--no-modern")
|| arg.equalsIgnoreCase("-nm")))
            ConsoleUtils.setModernMode(false);

        VirtualEscapePuzzle escapePuzzle = new VirtualEscapePuzzle();

        // populate with 1 room
        escapePuzzle.addRoom(new Room("The Tower", "What has hands but cannot hold a
thing?", "Clock"));

        printMenu();

        while (true) { if (!runGameLoopIteration(escapePuzzle)) break; }
    }

    /// @return true if the game loop should continue, false if it should end
    private static boolean runGameLoopIteration(VirtualEscapePuzzle escapePuzzle)
    {
        MenuOption option = getMenuOption();
        ConsoleUtils.oneLineSpace();

        switch (option)
        {
            case HELP -> printMenu();
            case CREATE_ROOM -> createRoom(escapePuzzle);
            case SOLVE_ROOM -> solveRoom(escapePuzzle);
            case VIEW_UNSOLVED_ROOMS -> viewRoom(escapePuzzle.getUnsolvedRooms(),
"Unsolved Rooms");
            case VIEW_SOLVED_ROOMS -> viewRoom(escapePuzzle.getSolvedRooms(),
"Solved Rooms");
            case VIEW_ALL_ROOMS -> viewRoom(escapePuzzle.getAllRooms(), "Rooms");
            case EXIT ->
            {
                ConsoleUtils.printBox(
                        "Goodbye!",
                        new String[]{"+ Thanks for playing the Virtual Escape
Puzzle!"},
                        GAME_BOX_WIDTH
                );
                return false;
```

```
            }
            default -> System.out.printf(
                    "> Invalid input. Please enter a number between %d and %d.%n",
                    MenuOption.firstIndex(),
                    MenuOption.lastIndex()
            );
        }

        ConsoleUtils.oneLineSpace();

        return true;
    }

    /* MENU OPTIONS */

    private static void printMenu()
    {
        final String[] options = MenuOption.getAllString();
        ConsoleUtils.printBox("Game Menu", options, HELP_BOX_WIDTH);
        ConsoleUtils.oneLineSpace();
    }

    private static void createRoom(VirtualEscapePuzzle escapePuzzle)
    {
        ConsoleUtils.printBox(
                "Room Creation",
                new String[]{"+ Answer the following questions to add a room"},
                GAME_BOX_WIDTH
        );
        ConsoleUtils.oneLineSpace();
        Room room = escapePuzzle.createRoom(
                getInputFromUser("Enter Room Name"),
                getInputFromUser("Enter Room Puzzle"),
                getInputFromUser("Enter Room Answer")
        );
        escapePuzzle.addRoom(room);
    }

    private static void solveRoom(VirtualEscapePuzzle escapePuzzle)
    {
        final ArrayList<String> options = new ArrayList<>();
        options.add("+ Enter the room name and your answer to solve a room");
        // populate options with unsolved room names
        options.addAll(escapePuzzle.getUnsolvedRooms()
                .stream()
                .map(Room::getRoomName)
                .map(name -> "  - " + name)
                .toList());

        ConsoleUtils.printBox("Solve Room", options.toArray(new String[0]),
GAME_BOX_WIDTH);

        ConsoleUtils.oneLineSpace();
        String roomName = getInputFromUser("Enter Room Name");
```

```
        Optional<Room> room = escapePuzzle.findRoomByName(roomName);
        if (room.isPresent())
        {
            System.out.println(room.get().getRoomPuzzle());
        }
        else
        {
            System.out.println("Room not found. Please check the room name and try
again.");
            return;
        }

        String roomAnswer = getInputFromUser("Enter Room Answer");
        switch (escapePuzzle.solveRoom(roomName, roomAnswer))
        {
            case WRONG_ANSWER -> System.out.println("Wrong answer. Try again.");
            case ROOM_NOT_FOUND -> System.out.println("Room not found.");
            case NO_ERROR -> System.out.println("Congratulations! You've solved the
room.");
        }
    }

    private static void viewRoom(ArrayList<Room> rooms, String title)
    {
        if (rooms.isEmpty())
        {
            System.out.printf("No %s found.%n%n", title);
            return;
        }
        ConsoleUtils.printBox(
                title,
                rooms.stream().map(room -> "+ " +
room.getRoomName()).toArray(String[]::new),
                GAME_BOX_WIDTH
        );
        ConsoleUtils.oneLineSpace();
    }

    /* HELPERS */

    private static String getInputFromUser(String prompt)
    {
        System.out.print(prompt + ": ");
        return IN.nextLine();
    }

    /* MENU SELECTION */

    private static MenuOption getMenuOption()
    {
        final int option;
        System.out.printf(
                "Select an option between %d and %d (%d for help): ",
                MenuOption.firstIndex(),
```

```
                MenuOption.lastIndex(),
                MenuOption.HELP.getIndex()
        );
        try { option = Integer.parseInt(IN.nextLine()); }
        catch (Exception e) { return MenuOption.INVALID; }
        return MenuOption.fromIndex(option);
    }

    enum MenuOption
    {
        HELP(0),
        CREATE_ROOM(1),
        SOLVE_ROOM(2),
        VIEW_UNSOLVED_ROOMS(3),
        VIEW_SOLVED_ROOMS(4),
        VIEW_ALL_ROOMS(5),
        EXIT(6),
        INVALID(-1);
        private final int index;

        MenuOption(int index) { this.index = index; }

        /* FACTORY */

        public static MenuOption fromIndex(int index)
        {
            for (MenuOption option : values())
                if (option.index == index) return option;
            return MenuOption.INVALID;
        }

        /* GETTERS & STATICS */

        public int getIndex() { return index; }

        public static int firstIndex() { return HELP.getIndex(); }

        public static int lastIndex() { return EXIT.getIndex(); }

        /* GET ALL */

        public static MenuOption[] getAll()
        {
            return Arrays.stream(values()).filter(option -> option !=
INVALID).toArray(MenuOption[]::new);
        }

        public static String[] getAllString()
        {
            // 1. getAll
            // 2. make all title format (only first capital)
            // 3. return
            return
Arrays.stream(getAll()).map(MenuOption::getAllFormat).toArray(String[]::new);
```

```
        }

        private static String getAllFormat(MenuOption option)
        {
            return option.getIndex() + ". " + option.name().charAt(0) +
option.name()
                    .substring(1)
                    .toLowerCase()
                    .replace("_", " ");
        }
    }
}
```

Program Output

```
┌──────────────────────────────┐
│      Game Menu               │
├──────────────────────────────┤
│ 0. Help                      │
│ 1. Create room               │
│ 2. Solve room                │
│ 3. View unsolved rooms       │
│ 4. View solved rooms         │
│ 5. View all rooms            │
│ 6. Exit                      │
└──────────────────────────────┘

Select an option between 0 and 6 (0 for help): 0

┌──────────────────────────────┐
│      Game Menu               │
├──────────────────────────────┤
│ 0. Help                      │
│ 1. Create room               │
│ 2. Solve room                │
│ 3. View unsolved rooms       │
│ 4. View solved rooms         │
│ 5. View all rooms            │
│ 6. Exit                      │
└──────────────────────────────┘


Select an option between 0 and 6 (0 for help): 1

┌──────────────────────────────────────────────────────┐
│           Room Creation                              │
├──────────────────────────────────────────────────────┤
│ + Answer the following questions to add a room       │
└──────────────────────────────────────────────────────┘

Enter Room Name: The Whispering Hallway
Enter Room Puzzle: What has keys but can't open locks?
Enter Room Answer: keyboard^[
```

```
Select an option between 0 and 6 (0 for help): 2

┌──────────────────────────────────────────────────────────┐
│             Solve Room                                   │
├──────────────────────────────────────────────────────────┤
│ + Enter the room name and your answer to solve a room    │
│   - The Tower                                            │
│   - The Whispering Hallway                               │
└──────────────────────────────────────────────────────────┘

Enter Room Name: the tower
What has hands but cannot hold a thing?
Enter Room Answer: abc
Wrong answer. Try again.

Select an option between 0 and 6 (0 for help): 2

┌──────────────────────────────────────────────────────────┐
│             Solve Room                                   │
├──────────────────────────────────────────────────────────┤
│ + Enter the room name and your answer to solve a room    │
│   - The Tower                                            │
│   - The Whispering Hallway                               │
└──────────────────────────────────────────────────────────┘

Enter Room Name: the tower
What has hands but cannot hold a thing?
Enter Room Answer: clock
Congratulations! You've solved the room.

Select an option between 0 and 6 (0 for help): 3

┌──────────────────────────────────────────────────────────┐
│          Unsolved Rooms                                  │
├──────────────────────────────────────────────────────────┤
│ + The Whispering Hallway                                 │
└──────────────────────────────────────────────────────────┘


Select an option between 0 and 6 (0 for help): 4

┌──────────────────────────────────────────────────────────┐
│           Solved Rooms                                   │
├──────────────────────────────────────────────────────────┤
│ + The Tower                                              │
└──────────────────────────────────────────────────────────┘


Select an option between 0 and 6 (0 for help): 5

┌──────────────────────────────────────────────────────────┐
│              Rooms                                       │
├──────────────────────────────────────────────────────────┤
│ + The Tower                                              │
```

```
| + The Whispering Hallway                    |
└─────────────────────────────────────────────┘


Select an option between 0 and 6 (0 for help): 2

┌─────────────────────────────────────────────┐
|            Solve Room                       |
├─────────────────────────────────────────────┤
| + Enter the room name and your answer to solve a room
|   - The Whispering Hallway                  |
└─────────────────────────────────────────────┘

Enter Room Name: the whispering hallway
What has keys but can't open locks?
Enter Room Answer: keyboard
Congratulations! You've solved the room.

Select an option between 0 and 6 (0 for help): 3

No Unsolved Rooms found.


Select an option between 0 and 6 (0 for help): 4

┌─────────────────────────────────────────────┐
|            Solved Rooms                     |
├─────────────────────────────────────────────┤
| + The Tower                                 |
| + The Whispering Hallway                    |
└─────────────────────────────────────────────┘


Select an option between 0 and 6 (0 for help): 5

┌─────────────────────────────────────────────┐
|              Rooms                          |
├─────────────────────────────────────────────┤
| + The Tower                                 |
| + The Whispering Hallway                    |
└─────────────────────────────────────────────┘


Select an option between 0 and 6 (0 for help): 0

┌──────────────────────────┐
|      Game Menu           |
├──────────────────────────┤
| 0. Help                  |
| 1. Create room           |
| 2. Solve room            |
| 3. View unsolved rooms   |
| 4. View solved rooms     |
| 5. View all rooms        |
```

```
| 6. Exit              |
└──────────────────────┘


Select an option between 0 and 6 (0 for help): 6

┌──────────────────────────────────────────────────────────────┐
|             Goodbye!             |                           |
├──────────────────────────────────────────────────────────────┤
| + Thanks for playing the Virtual Escape Puzzle!              |
└──────────────────────────────────────────────────────────────┘
```